

Optimizing Qwen2.5-Coder Inference on Tenstorrent Blackhole: Project Checkpoint

Rassul Magauin
Assylzhan Khamiyev
Mohammed Rashed Ali Yahmoor Alshehhi
Sultan Akimaliyev
Mohamed bin Zayed University of Artificial Intelligence
Abu Dhabi, UAE

Abstract

We report the first checkpoint of an inference optimization study of Qwen2.5-Coder on the Tenstorrent Blackhole p150 accelerator using the `tt-metal` SDK. Starting from the in-tree `tt_transformers` port with the stock Blackhole tuning recipe, we establish a device-level decode baseline on a four-device tensor-parallel mesh, identify the dominant kernels via the Tracy device profiler and the `tt-perf-report` utility recommended by the course staff, and rank three concrete optimization targets by projected return on engineering effort. Our measurements reveal that (i) the sampling-stage TopK operator accounts for 45–60% of per-token device kernel time but executes on a single Tensix core out of 130—an under-utilization artifact rather than a model bottleneck; (ii) the MLP feed-forward block is the largest *real* bottleneck at ~31% of device time, dominated by the gate/up and down projections; and (iii) the fused scaled-dot-product attention kernel is negligible (<1.5%), validating that existing flash-attention-style fusions on Blackhole already perform as intended. We propose parallelizing TopK, switching the MLP matmuls from HiFi4 to HiFi2 math fidelity, and fusing the gate/up projections as our next steps.

1 Introduction and Goals

Large language models tailored to code—such as Qwen2.5-Coder [1]—are increasingly deployed on non-GPU accelerators. Tenstorrent’s Blackhole p150 is an open-architecture, RISC-V-based tensor processor whose 130 worker Tensix cores each expose a matrix unit (FPU), a vector unit (SFPU), five Baby RISC-V CPUs, and 1.5 MB of L1 SRAM connected by a 2D NoC [2]. Unlike GPU stacks, `tt-metal` exposes kernels, circular buffers, and NoC scheduling directly to the developer, which creates both an optimization opportunity and a steep perf-debugging curve.

Project direction. Following the teaching staff’s redirect, we pursue *Direction B: Optimization*: we take the already-reported `tt_transformers` implementation of Qwen2.5 as a starting point and measure, rank, and remove bottlenecks

specific to Blackhole. A from-scratch port was rejected because `tt_transformers` already supports the Qwen2.5 family.

Contributions of this checkpoint.

1. A reproducible device-level profiling pipeline for `tt_transformers` decode on a four-device Blackhole mesh, combining the Tracy profiler with the `tt-perf-report` utility recommended by TA Dongning Ma.
2. A per-operator breakdown of Qwen2.5-Coder-7B-Instruct decode against the stock tuned recipe, distinguishing under-utilization artifacts (TopK) from real model bottlenecks (MLP).
3. A ranked list of three concrete, Blackhole-aware optimization targets with projected first-order ROI, to be executed and measured before the final report.

2 Background

Blackhole p150. Blackhole exposes an 8×10 compute grid (130 worker cores visible to `tt-metal`, compared to Wormhole’s 8×8), 1.5 MB of L1 per core, and eight DRAM banks. The in-tree `tt_transformers` framework gates several Blackhole-specific paths via `is_blackhole()`, most notably `prefill_len_cutoff` = 512 (vs. 1024 on Wormhole) and a Blackhole-only DRAM weight prefetcher currently wired for Llama-3.1-8B.

Qwen2.5-Coder-7B-Instruct. Qwen2.5-Coder-7B shares its base transformer with Qwen2.5-7B-Instruct: 28 layers, hidden size 3584, MLP intermediate 18944, 28 query heads with 4 grouped key/value heads, and `rope_theta` = 10^6 . This architectural parity lets us use the `performance_decoder_config.json` tuning recipe that `tt-metal` already ships for Qwen2.5-7B.

Fusion model of `tt-metal`. `tt-metal` fuses via circular-buffer choreography rather than monolithic kernels. A *reader* kernel streams tiles from DRAM or a neighbor core’s L1 into L1 circular buffers; a *compute* kernel drains them through the FPU/SFPU; and a *writer* kernel streams results out. Fusion is then a property of the CB topology, not a single file. We rely on five canonical fusion patterns already present in `tt-metal`: fused SDPA, `matmul+bias+activation`, distributed

Table 1. Per-iteration device-0 kernel time, Qwen2.5-Coder-7B-Instruct decode, 4-way Blackhole TP mesh, 10 layers, stock tuned recipe. TopK is a sampling-stage op, not a transformer-layer op.

Operator / group	Time (ms)	% of total
TopK (sampling, 1-core)	5.48	45.3
MLP gate+up projections	2.41	19.9
MLP down projection	1.40	11.5
Attention QKV projections	1.03	8.5
CCL (AllGather + ReduceScatter)	0.69	5.7
Layout/tiling/eltwise glue	0.46	3.8
Attention output projection	0.28	2.3
Fused SDPA decode + RoPE	0.15	1.3
RMSNorm	0.13	1.1
Other	0.08	0.6
Total per iter.	12.11	100.0

RMSNorm, async AllGather+matmul, and paged KV-cache update.

3 Methodology

Hardware. One machine with four Blackhole p150 devices at /dev/tenstorrent/{0..3}, running tt-metal built with ENABLE_TRACY=ON.

Workload. We drive the canonical models/tt_transformers/demo/simple_text_demo.py benchmark: decode mode, batch 1, 1024-token context, 10 transformer layers, paged attention on, and the stock performance. The blackhole-4 pytest fixture auto-selects a 4-way tensor-parallel mesh. Weights are local bf16 safetensors downloaded from Hugging Face.

Profiling pipeline. We wrap the pytest invocation with python -m tracy -p -r -a device_kernel_duration, which emits a per-op CSV under generated/profiler/reports/.../ops_perf_results_*.csv (4,916 rows across two trace-replay sessions, compile phase excluded). We post-process the CSV with two complementary tools: a small pandas script for per-op aggregates, and the tt-perf-report v1.2.2 utility distributed as a separate repository [3] (explicitly requested by our TA) for stacked per-op-class breakdowns and math-fidelity analysis.

4 Baseline Results

Table 1 reports the per-iteration device-kernel-time breakdown for Qwen2.5-Coder-7B-Instruct decode on a four-way Blackhole tensor-parallel mesh (device 0, steady-state trace replay, compile phase excluded). Total per-decode-iteration device kernel time is **12.1 ms** on the 7B-Instruct run and 10.4 ms on the Coder-7B run; the absolute delta is attributable to compile-cache warm-up between back-to-back runs (op counts and shapes are bit-identical, and the TopK kernel time matches to the nanosecond), so the *qualitative* ranking is robust across runs.

5 Bottleneck Analysis

TopK is an under-utilization artifact, not a layer bug. TopKDeviceOperation consumes 45–60% of per-token device kernel time across both profiled runs, with a shape of [1, 1, 32, 38016] and $k=32$. The kernel runs on a *single* Tensor six core out of the 130 available on Blackhole. This is not a transformer-layer bottleneck—it is a sampling-stage mapping bug that an existing multi-core reduction template should be able to remove nearly in full. We treat it as the highest-ROI, lowest-risk item on the list.

The MLP block is the real hot path. Excluding TopK, the MLP feed-forward block accounts for ~31% of device kernel time (≈ 3.81 ms/iter), split between the gate/up projections (3584 $\rightarrow$ 5120, 20 calls/iter) and the down projection (5120 $\rightarrow$ 3584). The single slowest individual op is the down projection at 139.8 μ s max. Per the tt-metal model-config database, these matmuls currently run in HiFi4 math fidelity on BF16/BFP8 operands.

Attention is already fused and cheap. The fused SDPA-decode kernel, head reshapes, RoPE, and KV-cache update together contribute only 1.3% of per-iter kernel time. This confirms that the flash-attention-style CB choreography in ttnn/cpp/ttnn/operations/transformer/sdpa/ is already doing what we would hope; attention is explicitly *not* a target.

Tensor-parallel overhead is modest. AllGather plus ReduceScatter (the TP tax) accounts for 5.7%. This is headroom worth chasing only after the MLP wins are realized.

6 Planned Optimizations and Projected ROI

We rank three candidates by (a) projected kernel-time recovery and (b) engineering effort.

- 1. Parallelize / fuse TopK.** Move TopKDeviceOperation off a single core onto a multi-core reduction template, or fuse it directly into SamplingDeviceOperation. Projected recovery: most of the 45–60% TopK slice. Effort: *low*—this is a dispatch-and-shard problem, not a new kernel. (Per-layer scaling: frees time regardless of layer count, since sampling is per-token.)
- 2. HiFi4 $\rightarrow$ HiFi2 for MLP matmuls.** The Blackhole FPU exposes a HiFi2 math-fidelity mode that trades the lowest activation bit for roughly 2 $\times$ matmul throughput. The tt-perf-report tool flagged this as the highest-impact single-line change available against the stock recipe. Projected recovery: up to half of the ~3.8 ms MLP slice (≈ 1.9 ms/iter, $\approx 15\%$ of TopK-excluded kernel time). Effort: *very low*—one field in performance_decoder_config.json. Risk: must verify with ci-token-matching that end-to-end generation quality is preserved.

3. **Fuse MLP gate+up projections.** Merge the two 3584 → 5120 projections into one 3584 → 10240 matmul, halving dispatch overhead and activation round-trips. Projected recovery: the second-largest slice after HiFi2. Effort: *medium*—requires checking whether the stock config already enables this and, if not, a model-code edit plus a perf/accuracy re-run.

7 Roadblocks and Risks

Hardware monitoring. `tt-smi` is not installed on our box; we confirm device health via first-program-launch runtime logs. Installing `tt-smi` is a next step.

Compile-cache variance. Back-to-back runs of architecturally identical models showed a ~44% per-op difference in Matmul mean time, attributable to compile-cache state. We will rerun each optimization experiment from a clean cache and report matched-cache deltas.

Accuracy regression from HiFi2. HiFi2 drops the lowest activation mantissa bit. We will gate the change behind the in-tree `ci-token-matching` parameterization and report token-match rates against the HiFi4 reference.

Coder-specific validation. Since Qwen2.5-Coder-7B is architecturally identical to Qwen2.5-7B-Instruct, our “Coder-ness” must be demonstrated at the accuracy/output level

(code-generation prompts), not at the kernel level. We will include a small code-completion token-match check in the final report.

8 Next Steps

Between now and the final report we will: (1) land the TopK multi-core fix and re-profile; (2) measure HiFi2 MLP speedup and verify token-match rate; (3) evaluate MLP gate+up fusion; (4) characterize single-device (non-TP) decode to isolate CCL overhead; and (5) profile prefill in addition to decode, since the Blackhole prefill-length cutoff of 512 tokens may expose a distinct set of hot paths.

References

- [1] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL]
- [2] Tenstorrent. 2026. TT-Metalium: Tenstorrent’s Low-Level Programming Model and SDK. <https://github.com/tenstorrent/tt-metal>. Accessed April 2026.
- [3] Tenstorrent. 2026. tt-perf-report: Performance Reporting Utility for TT-Metal Device Kernels. <https://github.com/tenstorrent/tt-perf-report>. Version 1.2.2, accessed April 2026.